

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	Materia: Programación Orientada a Objetos (POO404) Guia de Laboratorio No.2 Tema: Estructuras de decisión control y colecciones

I. OBJETIVOS.

Que el estudiante:

- Implemente los conceptos fundamentales de sentencias de control.
- Utilice las clases de Java para el manejo de las colecciones de datos.
- Utilice la sentencia Try Catch para capturar excepciones en tiempo de ejecución.

II. INTRODUCCIÓN.

Estructuras de control en Java

Todo lenguaje de programación cuenta (o al menos debería contar), con una serie de instrucciones que le permitan controlar el flujo de ejecución. Java consta de 2 tipos de dichas sentencias:

- ✓ Estructuras selectivas, en las cuales encontramos el if, else-if y switch.
- ✓ Estructuras repetitivas o ciclos, entre las cuales están while, do... while y for.

Estructuras selectivas

En la vida muchas veces es necesario elegir entre un camino y el otro a seguir. En muchas de las actividades que realizamos día con día, nos enfrentamos a decisiones que debemos tomar y que de una u otra forma alteran el cauce normal de nuestra vida (o de nuestro programa).

Como otros lenguajes, Java cuenta con estructuras para el control de flujo. Las decisiones se pueden realizar simples o anidadas(es decir, varias condiciones una dentro de otra).

Sentencia if

La sentencia if evalúa una condición y determina el curso a seguir; si la condición es falsa, se ejecuta el bloque else. La instrucción if tiene la siguiente estructura:

```
if (condición)
{
    //Código a ejecutar si condición es
    true
}
else
{
    //Código a ejecutar si condición es
    false
}
```

Esta instrucción evalúa la expresión **condición**, y si es **true**, ejecuta el código que hay entre las llaves que hay debajo de if. Si condición fuese false, el código a ejecutar sería el contenido entre las llaves que existen debajo de else.

El uso del bloque encabezado por else es opcional.

Una simplificación que se puede hacer es cuando sólo exista una instrucción dentro de las llaves (del if, o del else), en ese caso se pueden eliminar las llaves, es decir:

```
if (condición) //instrucción a ejecutar si condición es
true
    sentencia1;
else //instrucción a ejecutar si condición es false
    sentencia2;
```

Lo que no está permitido eliminar en ningún caso, son los puntos y coma (;) de las instrucciones que empleemos en la parte verdadera (if) o falsa (else), tengamos o no las llaves.

Toma de decisiones (Operadores)

Operadores Relacionales: Permiten comparar variables según relación de igual-dad/desigualdad o relación mayor/menor. Devuelven siempre un valor boolean.

• > : Mayor que • < : Menor que • == : Iguales	• != : Distintos • >= : Mayor o igual que • <= : Menor o igual que
--	--

Operadores Lógicos: Nos permiten construir expresiones lógicas.

• && : devuelve true si ambos operandos son true. • : devuelve true si alguno de los operandos es true. • ! : Niega el operando que se le pasa.	• & : devuelve true si ambos operandos son true, evaluándolos ambos. • : devuelve true uno de los operandos es true, evaluándolos ambos.
--	---

Sentencia switch

Permite seleccionar una opción de un conjunto de constantes. Si la variable indicada en switch coincide con la condición case, el bloque es ejecutado. Adicionalmente, la instrucción break finaliza el bloque de código case.

También existen ocasiones o programas donde se exige evaluar muchas condiciones a la vez, en estos casos, o se usa una condición compuesta muy grande o se debe intentar convertir el problema a uno que se pueda resolver usando la instrucción switch.

La instrucción switch es una instrucción de decisión múltiple, donde el compilador prueba o busca el valor contenido en una variable contra una lista de constantes (tipo int o chars).

Cuando el computador encuentra el valor de igualdad entre variable y constante, entonces ejecuta el grupo de instrucciones asociados a dicha constante, si no encuentra el valor de igualdad entre variable y constante, entonces ejecuta un grupo de instrucciones asociados a un default, aunque este último es opcional.

El formato de esta instrucción es el siguiente:

```
switch(var int o char)
{
    case const1: instrucción(es);
        break;
    case const2: instrucción(es);
        break;
    case const3: instrucción(es);
        break; .....
    default:
        instrucción(es);
};
```

Nota: Desde JDK 7, podemos usar una cadena literal/constante para controlar una declaración switch, lo cual no es posible en C/C++.

Usar un modificador basado en cadena/string es una mejora con respecto al uso de la secuencia equivalente if/else.

Estructuras iterativas

Los cálculos simples o la manipulación de pequeños conjuntos de datos se pueden realizar fácilmente a mano, pero las tareas grandes o repetitivas son realizadas con mayor eficacia por una computadora, ya que estas están especialmente preparadas para ello.

Para repetir varias veces un proceso determinado haremos uso de los ciclos repetitivos, a los cuales se les conoce con el nombre de estructura repetitiva, estructuras iterativas, lazo o bucles.

En C, al igual que en Java podemos encontrar 3 tipos de ciclos.

Ciclo while	Ciclo do while
<pre>while (condición) { conjuntoDeSentencias; }</pre>	<pre>do{ conjuntoDeSentencias; } while (condición);</pre>
Ciclo for (<i>dirigido por un contador</i>)	
<pre>for (inicialización, expresionBooleana, incremento) { conjuntoDeSentencias; }</pre>	

Bucle For Mejorado

Desde Java 5.0, se incluye un ciclo for hecho para recorrer colecciones con una sintaxis más simple.

Veamos su sintaxis:

```
for(tipo idParam : colección){  
    Sentencias;
```

- Necesitamos la **colección** por la cual iterar.
- **El tipo** no es más que la clase de los elementos que contiene esta colección.
- Finalmente necesitaremos también un **nombre de parámetro (idParam)** que representa el elemento de la colección que se va extrayendo en cada iteración.

Colecciones en Java

Una colección es un objeto que puede almacenar un número variable de elementos siendo cada elemento otro objeto.

Java tiene desde la versión 1.2 todo un juego de clases e interfaces para guardar colecciones de objetos. En él, todas las entidades conceptuales están representadas por interfaces, y las clases se usan para proveer implementaciones de esas interfaces.

Una interfaz nos dice qué podemos hacer con un objeto. Un objeto que cumple determinada interfaz es “algo con lo que puedo hacer X”. La interfaz no es la descripción entera del objeto, solamente un mínimo de funcionalidad con la que debe cumplir. Java organiza todas las clases de colecciones bajo el paquete **java.util**

• Collection

Una Collection es todo aquello que se puede recorrer (o iterar) y de lo que se puede saber el tamaño. Muchas otras clases extenderán la interfaz Collection imponiendo más restricciones y dando más funcionalidades.

La sintaxis para usar un collection es la siguiente:

```
Collection <clase_elementos> nombre_objeto =  
    new Collection<clase_elementos> ();
```

• List

La interfaz List define una sucesión de elementos donde cada uno de ellos lleva un índice asociado. Así, podríamos tener una lista con los nombres de las personas que han utilizado un servicio de acceso a internet que podría ser: usuarios --> (Jorge López, Andrea Maldonado, Luis Pérez, Elías Sánchez, Mirna Acosta). Donde cada contenido va asociado a un índice, usuario(0) sería Jorge López, usuario(1) sería Andrea Maldonado, usuario(2) sería Luis Pérez y así sucesivamente. Sintaxis:

```
List <clase_elementos> nombre_objeto = new List<clase_elementos> ();
```

• Map

Un Map representa lo que en otros lenguajes se conoce como “diccionario” y que se suele asociar a la idea de “tabla hash” (aunque no se implemente necesariamente con esa técnica). Un Map es

un conjunto de valores, con el detalle de que cada uno de estos valores tiene un objeto extra asociado. A los primeros se los llama “claves” o “keys”, ya que son los que permiten acceder a los segundos.

Un Map no es una Collection ya que esa interfaz es limitada. Podríamos decir que Collection es unidimensional, mientras que un Map es bidimensional. Su sintaxis es la siguiente:

```
Map <clase_claves, clase_valores> nombre_objeto =  
    new Map< clase_claves, clase_valores > ();
```

III. Procedimiento.

PARTE 1: Uso de sentencia condicional (if)

1. Cargue la aplicación IntelliJ-IDEA y cree un proyecto Java con el nombre **POOGuia2**.
2. Los siguientes programas nos permite utilizar la sentencia condicional **if**, digitar cada uno de ellos y si tiene alguna duda consultar a su docente.
3. Dentro de la carpeta **src**, cree un Paquete con el nombre **EjerciciosIF**. Luego, dentro del paquete anterior, cree una clase Java llamada **Controlif**.
4. Dentro del código fuente de la nueva clase, use la sentencia **import** para importar a la clase **java.util.Scanner**.
5. Defina al método estático **main**. Por ahora déjelo vacío (aun sin código).
6. Luego del cierre de la definición del método estático **main**, defina un nuevo método estático denominado **ejemplo1** con las siguientes líneas:

```
static void ejemplo1(){  
}
```

7. Dentro de las llaves del nuevo método **ejemplo1**, inserte la siguiente definición de código:

```
Scanner reader = new Scanner(System.in);  
int Var1,Var2;  
System.out.print("Ingrese numero1: ");  
Var1=reader.nextInt();  
System.out.print("Ingrese numero2: ");  
Var2=reader.nextInt();  
if(Var1==Var2)
```

```
System.out.print("Los numeros ingresados son iguales ");  
else  
    System.out.println("Los numeros ingresados No son iguales ");
```

8. Para ejecutar el código del método ejemplo1, ubique el cursor dentro del método main e invoque el método del paso anterior con esta línea:

```
ejemplo1( );
```

9. Ejecute la aplicación e ingrese 2 numeros iguales, para verificar el resultado devuelto. Vuelva a ejecutar la aplicación e ingrese a 2 numeros diferentes y observe el resultado.

10. Retorne a la definición de la clase **Controlif** e importe a la clase `javax.swing.JOptionPane`

11. Dentro de la clase, cree el siguiente método estático llamado **ejemplo2**, el cual no debe tener parámetros y no retornar valor (void):

```
static void ejemplo2(){  
  
}
```

12. Dentro de las llaves del método creado en el paso anterior, digite la siguiente definición:

```
String v1="";  
v1=JOptionPane.showInputDialog("Ingrese un valor");  
if(v1==null)  
    JOptionPane.showMessageDialog(null,"Apretaste cancelar");  
else {  
    if(v1.equals(""))  
        JOptionPane.showMessageDialog(null,  
            "No ingresaste nada");  
    else  
        JOptionPane.showMessageDialog(null,  
            "El valor de V1 es: "+v1);  
}  
System.exit(0);
```

13. Retorne a la definición de main y utilice a símbolos (`//`) para convertir en comentario a la llamada realizada al método **ejemplo1**.

14. Luego de esa línea, ahora incluya una llamada al método **ejemplo2**.
15. Vuelva a ejecutar la aplicación para probar el código del ejemplo 2. En el cuadro de dialogo, ingrese cualquier texto y de clic en *Aceptar*. Confirme el resultado devuelto en la consola.
16. Haga varias otras pruebas de ejecución y equivoquese a propósito, por ej. no ingrese valor y presione *Aceptar*, ingrese un valor pero presione *Cancelar*, no ingrese valor y presione *Cancelar*.

Estudie como puede identificar las condiciones del ingreso de datos capturados por un cuadro de dialogo de JOptionPane.

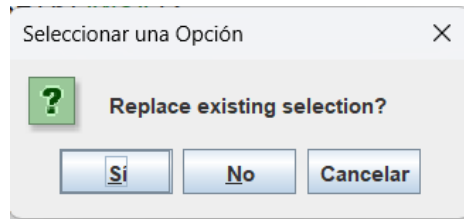
17. Luego del cierre de la definición del método **ejemplo2**, cree otro método estático bajo el nombre **ejemplo3**. Dentro de este nuevo método, digite las siguientes líneas de código:

```
static void ejemplo3(){
    int result = JOptionPane.showConfirmDialog(null,
        "Replace existing selection?");
    System.out.println(
        "El numero devuelto por showConfirmDialog es: "+result);

    if (result == JOptionPane.YES_OPTION)
        System.out.println("Yes");
    else if (result == JOptionPane.NO_OPTION)
        System.out.println("No");
    else if (result == JOptionPane.CANCEL_OPTION)
        System.out.println("Cancel");
    else if (result == JOptionPane.CLOSED_OPTION)
        System.out.println("Closed");

    System.exit(0);
}
```

18. Retorne al método main, ignore la llamada al método **ejemplo2** e inserte la llamada al **ejemplo3**.
19. Ejecute varias veces a la nueva aplicación y estudie la manera como showConfirmDialog detecta la elección del botón presionado por el usuario.



20. Guarde los cambios realizados hasta ahora, haciendo clic en el menú *File -> Save All* o presionando las teclas **Ctrl + S**

21. Retorne al código de la clase y cree un último método estático, bajo el nombre **ejemplo4**.

Luego, dentro de este método le dará solución al siguiente problema:

“Diseñe un programa en java que calcule los descuentos a un trabajador sabiendo que son aplicables un 6.25% del salario en AFP solo si este es superior a \$300.00. Además, si es un trabajador hombre se le descuenta aparte del AFP, el 3% sobre el sueldo en conceptos de ISSS y 10% en concepto de RENTA”

22. Digite la siguiente definición a este nuevo método **ejemplo4** :

```
String datos;
double sueldo, afp=0, isss, totalre, nsueldo, renta;
datos = JOptionPane.showInputDialog(" Ingrese el sueldo del empleado (a)");
sueldo = Double.parseDouble(datos);
if(sueldo>300)
    afp = (sueldo*0.0625);

JOptionPane.showMessageDialog(null,
    " Este empleado tiene un sueldo de " + sueldo +
    " y el descuento del AFP es "+ afp);
datos= (String) JOptionPane.showInputDialog(null,
    "Ingrese el Sexo:\nSi es Masculino (M)\nSi es Femenino (F)",
    "Sexo del Empleado",JOptionPane.QUESTION_MESSAGE,
    null, //Icono por defecto
    new Object[] { "M", "F"},
    "F"); //opcion por defecto
if(datos == "M")//Si es de sexo Masculino
{
    isss = sueldo*0.03;
    renta=sueldo*0.10;
    totalre = afp+isss+renta;
    nsueldo = sueldo - totalre;
    JOptionPane.showMessageDialog(null,
        "A este empleado se le detiene" + isss+
        "en concepto de ISSS\nAdemas se le retiene: "+ renta+
```

```

        "En concepto de Renta\nLo que hace un total de "+
        totalre+ "\nY su nuevo Sueldo es de:"+ nsueldo);
    }

```

23. De manera similar a los ejemplos anteriores, retorna a main, comente la llamada del método **ejemplo3** e inserte la llamada al método **ejemplo4**.
24. Ejecute la aplicación y brinde valores de prueba para los datos solicitados ahí.
25. Guarde los cambios realizados hasta ahora en su proyecto.

PARTE 2: Uso de sentencia selectiva (switch)

26. En la ventana de la estructura de su proyecto, localice la carpeta src y cree un paquete llamado **EjerciciosSwitch**. Dentro de este paquete, cree una clase llamada **Controlswitch**.
27. Dentro de la nueva clase, redacte una definición para el método estático **main**.
28. Luego de main, declare a otro método estático llamado **switch1** con el siguiente encabezado:

```
public static void switch1(int a, int b, char op)
```

29. En el interior del método del paso anterior, redacte el siguiente código:

```

switch ( op ) {
    case '+':
        //imprime mensaje como concatenacion de varios String
        System.out.println("El resultado de "+
            a+" "+op+" "+b + " es "+(a+b));
        break;
    case '-':
        //imprime mensaje basado en una cadena String con formato,
        //identica a como se hace con C++
        // %n: genera efecto de \n
        System.out.printf("El resultado de %d %c %d es %d %n",
            a, op, b, a-b);
        break;
    case '*':
        // imprime signo de cada valor numerico
        // Signo se mostrara aunque valor sea positivo
        System.out.printf("El resultado de %+d %c %+d es %d %n",
            a, op, b, a * b);
        break;
}

```

```

    case '/':
        // Se aplica conversion explicita a variable b, para
        // operar division como resultado decimal
        System.out.printf("El resultado de %d %c %d es %.4f%n",
            a, op, b, a/(double)b);
        break;
    default:
        System.out.printf(
            "Error, simbolo (%c) no es un operador aritmetico",
            op);
        break;
} //fin switch op

```

30. Retorne al método main e invoque al método switch1, enviando como argumento a 4, -9 y '+', con esta línea:

```
switch1(3, -9, '+'); //invoca a metodo switch1
```

31. Ejecute el programa y confirme que imprime la suma de ambos valores.
32. Cambie el argumento del carácter '+' por el de resta '-' y verifique que imprime la resta.
33. Repita el paso anterior para ejecutar la multiplicación (con '*') y la división (con uso de '/')
- Observe las modificaciones de presentación ejecutadas por los operadores de formato.
34. Haga una ultima prueba con un carácter que no sea un operador matemático, como por ej. '&

35. Para continuar, importe en la clase actual a la clase **JOptionPane**.

36. Proceda a crear un método estático con el nombre **switch2**. Luego, digite la siguiente definición para este método:

```

String result = "?";
int botonpresionado = JOptionPane.showConfirmDialog(null,
    "Quiere reemplazar su seleccion?");

switch (botonpresionado) {
    case JOptionPane.CANCEL_OPTION:
        result = "Canceled";
        break;
    case JOptionPane.CLOSED_OPTION:
        result = "Closed";

```

```

        break;
    case JOptionPane.NO_OPTION:
        result = "No";
        break;
    case JOptionPane.YES_OPTION:
        result = "Yes";
        break;
    }
    System.out.println(
        "Boton elegido del cuadro de dialogo: " + result);

```

37. Localice al método main e ignore la llamada al método **switch1**. A cambio, invoque al método **switch2**.
38. Ejecute la aplicación y verifique que le identifica el botón presionado en un cuadro de dialogo de confirmación.
39. Guarde los cambios hechos en su proyecto.

PARTE 3: Uso de bucles/ciclos

40. Cree un nuevo paquete, con el nombre **EjerciciosBucles**. Luego, en su interior, cree a una clase llamada **CiclosJava**.
41. Importe a la clase JOptionPane. Además, defina al método estático main.
42. Cree a 2 métodos estáticos, uno con el nombre **factorial** y el otro como **ciclos1**.
Ambos no retornan valor al final (void). Y al método **factorial**, incluirle un parámetro entero llamado **n**.
43. Definir el siguiente código al método **factorial**:

```

long resultado=1;
for(int i=1; i<=n; i++){
    resultado=resultado*i;
    System.out.printf("%d ", resultado);
}
JOptionPane.showMessageDialog(
    null, "El factorial de "+n+" es: " +
    resultado, "Resultado",
    JOptionPane.INFORMATION_MESSAGE);
System.out.printf("\nValor de %d! es %d",n,resultado);

```

44. Digite el siguiente cuerpo de código al método **ciclos1**:

```
int numero;
int valor;
System.out.println(
    "Programa para calculo del factorial de un numero\n");
String strnumero = JOptionPane.showInputDialog(
    "Ingrese Numero entero: ");
numero=Integer.parseInt(strnumero);
factorial(numero);
```

45. Ubique al método main y en su interior invoque al método **ciclos1**.

46. Ejecute la aplicación e ingrese el valor 7. La ejecución le devolverá que el factorial de 7 es de 5040.

47. Ahora se dará solución al siguiente problema:

En una empresa, se tienen datos correspondientes a los sueldos de 5 empleados, de los cuales, se desea saber, quien goza del sueldo mayor, quien goza del sueldo menor y cuantos poseen un sueldo mayor de \$300.00.

48. Dentro de la clase **CiclosJava**, cree un método estático, con el nombre **ciclos2**. Dentro de este método, digite la siguiente definición de código:

```
String leer;
double sueldo, mayor=0, menor=0;
int i=1, contador=0;

while(i<=5)
{
    leer= JOptionPane.showInputDialog(
        "Ingrese el Sueldo del Empleado " +i);
    sueldo= Double.parseDouble(leer);
    if(sueldo>300) contador= contador + 1;
    if(sueldo>mayor) mayor=sueldo;
    if(menor==0) menor = sueldo;
    if(sueldo<menor) menor=sueldo;
    i++;
}
```

```

JOptionPane.showMessageDialog(null,
    " El sueldo Mayor es de $: " + mayor +
    "\nEl sueldo Menor es de $: " + menor +
    "\n"+ contador +
    " Empleados tienen un sueldo mayor de $300");
}

```

49. Dentro del método main, comente la llamada al método **ciclos1**. A cambio, incluya la llamada al método **ciclos2**.

50. Pruebe la aplicación e ingrese los sueldos de 5 empleados. Confirme los resultados devueltos.

51. Guarde los cambios en su proyecto java.

52. Cree un método estático, bajo el nombre **ciclos3** y con la siguiente definición:

```

static void ciclos3(){
    //declara e inicializa a un vector
    int Numero[]={1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
    int Suma=0; //acumulador
    //for clasico
    for(int indice=0; indice<10; indice++){
        //acumula suma de valores
        Suma = Suma + Numero[indice];
    }
    System.out.println("Total con for clasico " + Suma);

    Suma=0; //reinicializa acumulador de suma de valores

    //utilizacion de for mejorado
    for(int valor: Numero){
        Suma += valor;
    }
    System.out.println("Total con for mejorado " + Suma);
}

```

53. Retorne a main, ignore la llamada del ultimo método comprobado e inserte la llamada al método creado en el paso anterior.

54. Ejecute la aplicación y compruebe el funcionamiento del ciclo for y del ciclo for mejorado, este ultimo orientado al recorrido de vectores y colecciones en Java.

PARTE 4: Cuadros de dialogo

55. Cree un paquete y guárdelo con el nombre **CuadrosDialogos**.

Dentro de este paquete cree a una clase llamada **Dialogos**. Dentro de la clase defina al método estatico **main** y además, importe a la clase **JOptionPane**

56. Cree un método estático llamado **dialogo1** y digite ahí al siguiente código:

```
while (true){
    try{
        String nombre= JOptionPane.showInputDialog(
            "Como te llamas?");
        String entrada= JOptionPane.showInputDialog(
            "cuántos años tienes");
        int edad=Integer.parseInt(entrada);
        JOptionPane.showMessageDialog(null,"Hola, "+
            nombre + ". El año que viene tendrás " +
            (edad+1) + "años",
            "Resultado",JOptionPane.INFORMATION_MESSAGE);
    }
    catch(Exception e){
        System.out.println("Error en edad ingresada");
        JOptionPane.showMessageDialog(null,
            "Error en algun dato de entrada",
            "Error",JOptionPane.ERROR_MESSAGE);
        String seleccion=(String) JOptionPane.showInputDialog(
            null, "Desea Salir", "Seleccione una opcion",
            JOptionPane.QUESTION_MESSAGE,
            null, // null para icono defecto
            new Object[] { "Si", "No" },
            "Si");
        if (seleccion.equals("Si")){
            System.exit(0);
            break;
        }
    }
} //fin de catch
} //fin de while
```

57. Retorne a la definición de main e invoque ahí al método **dialogo1**.

58. Proceda a ejecutar la aplicación e ingrese unas 3 parejas de nombres y edades. A la 4ta vez, deje la edad en blanco. Se genera una excepción y la aplicación le solicita si finalizar su ejecución o continuar. Marque el botón No.
59. La ejecución se encuentra en un ciclo infinito. Esta vez, deje vacío el nombre y cuando solicite la edad, escriba texto. Se genera de nuevo una excepción. Esta vez presione el botón Si del cuadro de dialogo.

Tipos de iconos en JOptionPane

60. Para continuar, se mostrarán los diferentes tipos de cajas de dialogo para mostrar mensajes.
61. Cree un método estático y asígnele como nombre a **dialogo2**. Digite el siguiente código en su definición:

```
//Prueba de uso de titulo e icono por defecto en un cuadro de dialogo
JOptionPane.showMessageDialog(null,"Informacion",
    "Mensaje De Informacion",JOptionPane.INFORMATION_MESSAGE);
JOptionPane.showMessageDialog(null,"Advertencia",
    "Mensaje de Advertencia",JOptionPane.WARNING_MESSAGE);
JOptionPane.showMessageDialog(null,"Error",
    "Mensaje de Error",JOptionPane.ERROR_MESSAGE);
JOptionPane.showMessageDialog(null,"Sin Icono",
    "Mensaje de Texto Plano",JOptionPane.PLAIN_MESSAGE);
```

62. Retorne a main y comente la llamada al método **dialogo1**. Redacte la llamada al método **dialogo2**.
63. Ejecute nuevamente la aplicación.
- Identifique el tipo de icono usado para clasificar al tipo de mensaje que puede ser presentado en un cuadro de mensaje de dialogo (MessageDialog).

Resultados





64. Guarde los cambios realizados hasta ahora.

PARTE 4: Manejo de Colecciones

65. Cree un paquete con el nombre **Colecciones**. Luego, en su interior cree a la Clase **UsoColecciones**.

66. En la nueva Clase, importe a las clases `java.util.ArrayList` y `java.util.Collection`

67. Dentro de la Clase `UsoColecciones`, cree a un método estático **main**.

68. Cree a otro método estático bajo el nombre **ejemploarraylist**. Luego, digite el siguiente código como definición para este metodo:

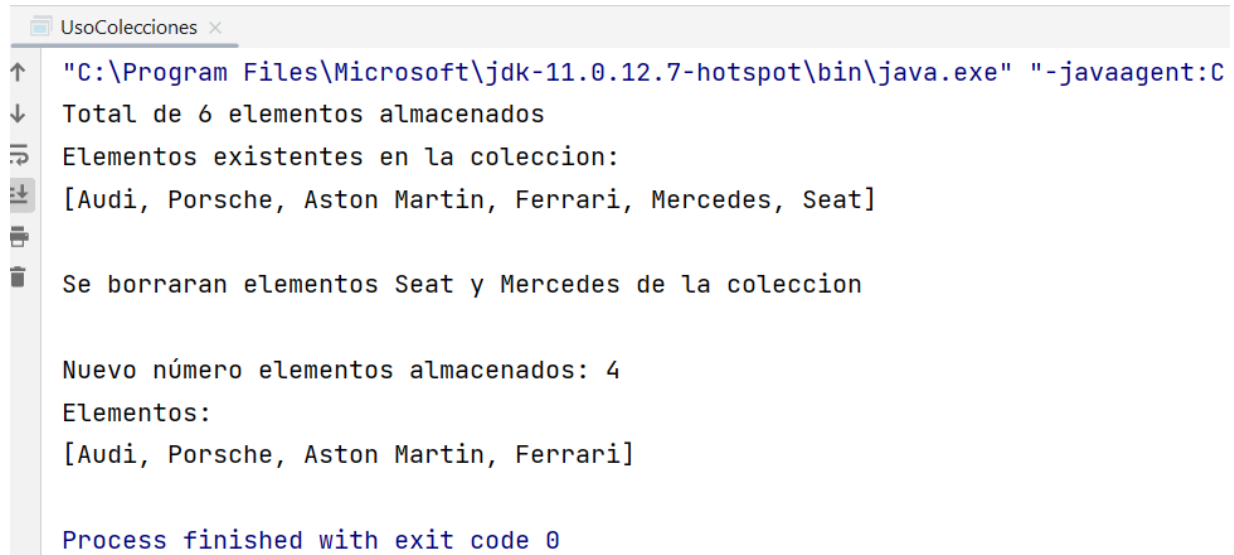
```
//listaMarcasCoches sera objeto Collection de la clase ArrayList
Collection listaMarcasCoches = new ArrayList<String>();
//Agregando elementos al objeto
listaMarcasCoches.add("Audi");
listaMarcasCoches.add("Porsche");
listaMarcasCoches.add("Aston Martin");
listaMarcasCoches.add("Ferrari");
listaMarcasCoches.add("Mercedes");
listaMarcasCoches.add("Seat");
System.out.printf("Total de %d elementos almacenados%n",
    listaMarcasCoches.size() );//Obtiene tamaño de Collection

//devuelve lista de elementos almacenados
System.out.println ("Elementos existentes en la Coleccion:\n"
+listaMarcasCoches.toString() );
System.out.println("\nSe borrarán elementos Seat y Mercedes de la
coleccion\n");
listaMarcasCoches.remove ("Seat"); //Removiendo elemento por nombre
listaMarcasCoches.remove ("Mercedes"); //Removiendo elemento por nombre
System.out.println("Nuevo número elementos almacenados: " +
```

```
listaMarcasCoches.size() );  
System.out.println("Elementos:\n" + listaMarcasCoches.toString ( ) );
```

69. Dentro del método main, invoque al método **ejemploarraylist**.

70. El resultado devuelto sera el siguiente:



```
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe" -javaagent:C  
Total de 6 elementos almacenados  
Elementos existentes en la coleccion:  
[Audi, Porsche, Aston Martin, Ferrari, Mercedes, Seat]  
  
Se borrarán elementos Seat y Mercedes de la coleccion  
  
Nuevo número elementos almacenados: 4  
Elementos:  
[Audi, Porsche, Aston Martin, Ferrari]  
  
Process finished with exit code 0
```

71. Al inicio del archivo de la clase, importe a las clases `java.util.Arrays` y luego a `java.util.Iterator`

72. Cree un nuevo método estático, con el nombre **ejemplolist** y con la siguiente definición:

```
static void ejemplolist(){  
  
    ArrayList <String> listaPersona = new ArrayList();  
  
    //Agregando elementos  
    listaPersona.add("Marie Curie");  
    listaPersona.add("Benjamin Franklin");  
    listaPersona.add("Marco Antonio");  
    listaPersona.add(1, "Juan Murillo");  
  
    //Mostrar contenido de ArrayList  
    System.out.println("USO DE ARRAYLIST");  
    System.out.println("Contenido de listaPersona: " +  
        listaPersona + "\n");  
    //Obteniendo posición de un elemento  
    int pos = listaPersona.indexOf("Benjamin Franklin");  
    System.out.println("El indice de Benjamin Franklin es: " + pos);  
}
```

```

//Verificando si lista está vacía
boolean check = listaPersona.isEmpty();
System.out.println("Verificando si ArrayList está vacío: " + check);

//Obteniendo tamaño de lista
int size = listaPersona.size();
System.out.println("El tamaño de listaPersona es: " + size);
//Verificando si un elemento está incluido en la lista
boolean elemento = listaPersona.contains("Marco Antonio");
System.out.println(
    "Verificando si 'Marco Antonio' está incluido en listaPersona: " +
    elemento);

//Obteniendo elemento de una posición específica
String item = listaPersona.get(0);
System.out.println("El elemento con índice 0 es: " + item);

//Recuperando elementos del ArrayList

//Primera forma: Usando Lazo for
System.out.println("\nRecuperando elementos del ArrayList con FOR");
for (int i = 0; i < listaPersona.size(); i++) {
    System.out.println("Índice: " + i + " - Elemento: " +
        listaPersona.get(i));
}

//Segunda forma: Usando Lazo foreach
System.out.println(
    "\nRecuperando elementos del ArrayList con FOREACH");
for (String persona : listaPersona) {
    System.out.println("Elemento: " + persona);
}

//Tercera forma: Usando Iterator
//hasNext(): devuelve true si hay mas elementos
//next(): devuelve el siguiente elemento
System.out.println(
    "\nRecuperando elementos del ArrayList con ITERATOR");

for (Iterator<String> it = listaPersona.iterator(); it.hasNext();)
{
    System.out.println("Elemento: " + it.next());
}

//Reemplazando un elemento
listaPersona.set(1, "Nikola Tesla");
System.out.println("\nArrayList después de reemplazo:\n" +
    listaPersona);

```

```

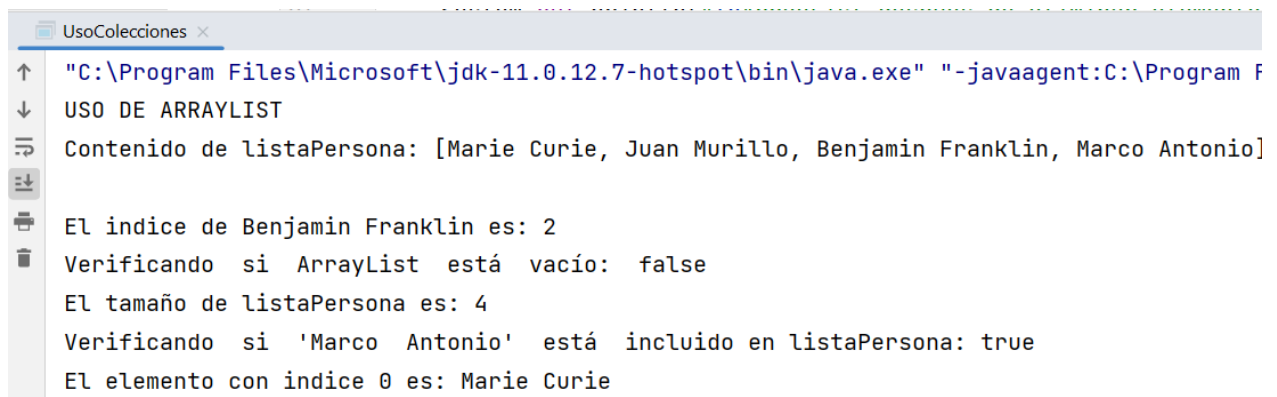
//Eliminando elemento en posicion 0
listaPersona.remove(0);
System.out.println("\nArrayList después de eliminar elemento 0:\n" +
                    listaPersona);

//Convirtiendo ArrayList en Array
String[] simpleArray = listaPersona.toArray(
                                new String[listaPersona.size()] );
System.out.println("\nObjeto Array creado es:\n" +
                    Arrays.toString(simpleArray));
}

```

73. Dentro del método **main**, convierta en comentario la llamada al método **ejemploarraylist** e inserte la llamada al método **ejemplolist**.

74. Ejecute nuevamente la aplicación. El resultado devuelto sera el siguiente:



```

UsoColecciones x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe" "-javaagent:C:\Program F
USO DE ARRAYLIST
Contenido de listaPersona: [Marie Curie, Juan Murillo, Benjamin Franklin, Marco Antonio]

El indice de Benjamin Franklin es: 2
Verificando si ArrayList está vacío: false
El tamaño de listaPersona es: 4
Verificando si 'Marco Antonio' está incluido en listaPersona: true
El elemento con indice 0 es: Marie Curie

Recuperando elementos del ArrayList con FOR
Indice: 0 - Elemento: Marie Curie
Indice: 1 - Elemento: Juan Murillo
Indice: 2 - Elemento: Benjamin Franklin
Indice: 3 - Elemento: Marco Antonio

Recuperando elementos del ArrayList con FOREACH
Elemento: Marie Curie
Elemento: Juan Murillo
Elemento: Benjamin Franklin
Elemento: Marco Antonio

ArrayList después de reemplazo:
[Marie Curie, Nikola Tesla, Benjamin Franklin, Marco Antonio]

ArrayList después de eliminar elemento 0:
[Nikola Tesla, Benjamin Franklin, Marco Antonio]

```

Objeto Array creado es:

[Nikola Tesla, Benjamin Franklin, Marco Antonio]

Process finished with exit code 0

75. Del paquete `java.util`, importe a las clases `HashMap`, `Map` y por último a `Set`.

76. Cree a un método estático, con el nombre **usodemap**. Dentro de este método, defina el siguiente código:

```
static void usodemap(){
    //Declarando HashMap
    HashMap<Integer, String> hmap = new HashMap<Integer, String>();

    //Agregando elementos
    hmap.put(12, "Crayolas");
    hmap.put(2, "Lápices");
    hmap.put(7, "Borradores");
    hmap.put(49, "Colores");
    hmap.put(3, "Boligrafos");

    //Mostrando contenido usando Iterator
    System.out.println("Contenido de HashMap:");
    Set set = hmap.entrySet();
    Iterator iterador = set.iterator();
    while (iterador.hasNext()) {
        Map.Entry mentry = (Map.Entry) iterador.next();
        System.out.println("Clave: " + mentry.getKey() +
            " - Valor: " + mentry.getValue());
    }
    //Obtener valor en base a clave
    String var = hmap.get(2);
    System.out.println("\nValor asociado a clave 2: " + var);

    //Remover elementos en base a clave
    hmap.remove(3);
    System.out.println(
        "\nHashMap después de eliminar elemento con clave 3:");

    Set set2 = hmap.entrySet();
    Iterator iterador2 = set2.iterator();

    while (iterador2.hasNext()) {
        Map.Entry mentry2 = (Map.Entry) iterador2.next();
        System.out.println("Clave: " + mentry2.getKey() +
            " - Valor: " + mentry2.getValue());
    }
} //fin de metodo usodemap
```

77. Retorne a **main** e ignore la llamada al método **ejemplolist**. Ahora agregue la llamada al método **usodemap**.

78. Ejecute la aplicación. El resultado sera el siguiente:

```
UsoColecciones x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe"
Contenido de HashMap:
Clave: 49 - Valor: Colores
Clave: 2 - Valor: Lápices
Clave: 3 - Valor: Boligrafos
Clave: 7 - Valor: Borradores
Clave: 12 - Valor: Crayolas

Valor asociado a clave 2: Lápices

HashMap después de eliminar elemento con clave 3:
Clave: 49 - Valor: Colores
Clave: 2 - Valor: Lápices
Clave: 7 - Valor: Borradores
Clave: 12 - Valor: Crayolas

Process finished with exit code 0
```

79. Guarde los cambios en el proyecto y cierrelo.

V. EJERCICIOS COMPLEMENTARIOS.

1. Dentro de la clase Controlif del procedimiento, ubique el método ejemplo3. Convierta en comentario al bloque de líneas de las estructuras if implementadas ahí y luego, redacte un código que reemplace a todos los if por solamente una estructura selectiva switch.
2. Elabore un metodo que recibe en parametros a los coeficientes (a, b, c) de una ecuacion cuadratica e imprime en pantalla a las raices de esta ecuacion. La solucion puede ser real, imaginaria o compleja. Debe crearse un metodo complementario que retorne el determinante de una ecuacion cuadrática, cuyos coeficientes los recibe en parametros.

Algunas ecuaciones que debería poder resolver el método son las siguientes:

$$16x^2 + 0x + 9 = 0$$

$$1x^2 - 2x + 5 = 0$$

$$4x^2 - 4x + 7 = 0$$

3. Una tienda implementa la siguiente politica de descuentos sobre la compra de sus clientes:
Cuando el cliente cancela su compra en la caja, el sistema le genera aleatoriamente una bolita y dependiendo del color, se le aplica un porcentaje de descuento. Si la bolita generada es cafe, se le aplica un 10% de descuento sobre la compra; si la bola es roja, se le aplica un 15% de descuento, un descuento del 25% si es azul y si la bolita es verde, tendrá el 50% de descuento.
Elabore un metodo estático en java que reciba en parametros al nombre de un cliente y el monto de la compra a realizar. Luego, este metodo ejecuta la politica de descuentos descrita en este ejercicio.
4. Desarrolle una aplicacion que utilice una coleccion HashMap para administrar titulos de libros para una biblioteca. Cada libro por registrar tiene un ISBN y un titulo.
La administración debe implementar las 4 acciones definidas en un CRUD y la opcion finalizar. El objeto de la coleccion HashMap se declara como un campo estatico en la clase y cada accion del CRUD se realiza dentro de un metodo distinto.
Para identificar si un ISBN esta bien escrito, investigue acerca del uso de *expresiones regulares* en java.

Cada vez que un usuario se equivoque al ingresar un dato, el programa lo vuelve a solicitar hasta ingresarlo correctamente. Usuario puede equivocarse hasta 3 veces, de lo contrario, se cancela la operacion y se retorna al menu principal.

VI. REFERENCIA BIBLIOGRAFICA.

Libros en la biblioteca de la UDB:

- Fain, Yakov. 2011. Programación Java. Madrid: Anaya Multimedia.
- Vegas Gertrudix, J. M. (2022). Java 17 : programación avanzada. Ra-Ma. <http://www.digitaliapublishing.com/a/116382/>
- Groussard, T. (2013). Java 7 : bases del lenguaje y de la programación orientada a objetos. ENI.